

Helix Thready — Deployment & Operations (Area Index)

Table of Contents

1. Scope & operator decisions
2. Upstream / Downstream dependencies
3. In-house modules this area binds to
4. Deployment overview diagram
5. Document map
6. Gap-register items addressed by this area
7. Open items tracked by this area
8. Verified vs assumed — reading guide

Rev	Date	Author	Change
1	2026-07-21	swarm (deployment)	Initial implementation-ready draft of the full Deployment & Operations pack
2	2026-07-21	swarm (deployment review)	Review pass: added OpenAPI 3.1 health contract + reproduce-first TDD skeletons, fixed the boot <code>--wait</code> contradiction, addressed GAP #17, split single-paragraph diagram explanations
3	2026-07-22	swarm (deployment)	Pass 3 depth: added <code>compose-files.md</code> (complete per-env rootless compose YAML) + <code>operations-runbook.md</code> (day-2 ops & incident playbooks); re-verified every module signature at <code>vasic-digital/containers</code> + <code>vasic-digital/lets_encrypt</code> source; narrowed <code>[OPEN: dns-provider]</code> (Loopia/ <code>dns_loopia</code>) and <code>[OPEN: host-sizing]</code> (candidate Hetzner SKUs)
4	2026-07-22	swarm (deployment critic)	Completeness pass: added <code>cost-and-capacity.md</code> (closes the §16.3 “quantify in the deployment pack” promise — host/object/backup/GPU cost + subscription/metered offset model) and <code>monitoring-observability.md</code> (Q42/Q43 depth — SLOs, the service/edge/cert/latency/resource alert rules the runbook referenced but were undefined, routing, dashboards, reproduce-first alert test); completed the deterministic port plan (ClickHouse native <code>9009</code> , Jaeger OTLP <code>4317</code>) to match the compose files

This is the canonical entry point for the **Deployment & Operations** documentation of Helix Thready. It specifies how the system is provisioned, containerized, secured with TLS, released, rolled back, backed up and recovered on a **single Hetzner dedicated host**, running **three fully-separated environments** (`dev` /

`sta` / `prod`) as **rootless Podman Compose** stacks behind subdomains, using only the in-house `basic-digital` module fleet named in the technology decision matrix. All authors follow **CONVENTIONS.md**.

Every Mermaid diagram in this pack has a sibling source under `diagrams/` and a following multi-paragraph prose explanation. Rendered PNG/SVG exported via Docs Chain (§11.4.65).

Table of Contents

1. Scope & operator decisions
 2. Upstream / Downstream dependencies
 3. In-house modules this area binds to
 4. Deployment overview diagram
 5. Document map
 6. Gap-register items addressed by this area
 7. Open items tracked by this area
 8. Verified vs assumed — reading guide
-

1. Scope & operator decisions

The scope of this area is fixed by the operator decisions in the final research request (§0.1) and answers Q8, Q21, Q39, Q41–Q45 `[OPERATOR]` `[CONSTITUTION]` :

Concern	Decision	Provenance
Orchestration	Rootless Podman Compose only via <code>vasic-digital/containers</code> — no Docker, no <code>sudo</code>	[CONSTITUTION §11.4.76/161]
Host	One Hetzner dedicated host ; root provisions the <code>thready</code> user (<code>/home/thready</code>)	[OPERATOR] [CONSTITUTION §11.4.76]
Environments	Three fully-separated stacks behind subdomains (<code>dev.</code> / <code>sta.</code> / <code>thready.</code>)	[OPERATOR] (Q8, §8.2)
TLS	<code>vasic-digital/lets_encrypt</code> (acme.sh, HTTP-01/DNS-01, atomic deploy-hook + rollback)	[IN-HOUSE: lets_encrypt] (Q44)
Ports / discovery	Dynamic ports via <code>port_prefix</code> , discovery via <code>discovery</code> + <code>mdns</code>	[IN-HOUSE] (§14.3)
CI/CD	No server-side CI — local git-hooks + pre-tag full-suite retest + all-upstreams push	[CONSTITUTION §11.4.156/75/40/§2.1] (Q21)
Secrets	Runtime-load-only from gitignored <code>.env</code> / <code>api_keys.sh</code> , <code>chmod 600/700</code> , never logged	[CONSTITUTION §11.4.10] (Q39)
Backup / DR	Daily full + hourly DB incrementals ; RPO ≈ 1 h, RTO ≈ 4 h; documented restore runbook	[OPERATOR] (Q41, Q45)
Monitoring	Prometheus + Grafana + OpenTelemetry (Jaeger) + ClickHouse via <code>observability</code>	[IN-HOUSE] (Q42/Q43)

The three subdomains (from §8.2):

Environment	Domain	Purpose
Development	<code>dev.thready.hxd3v.com</code>	Development testing
Staging	<code>sta.thready.hxd3v.com</code>	Pre-production
Production	<code>thready.hxd3v.com</code>	Live system

2. Upstream / Downstream dependencies

Upstream (this area consumes):

- **architecture** — the service inventory (Herald, Processing Engine, Asset Service, Semantic Search, API, Event Bus) that the container topology packages.
- **database** — the PostgreSQL + pgvector schema, migration runner (`migration.Runner`) and partitioning strategy that the deploy/rollback and backup runbooks operate on (expand-contract migrations; PITR from WAL).
- **api** — the REST `/v1` (HTTP/3) + WebSocket/SSE surface the edge reverse proxy fronts and the `LE_VALIDATE_URL` health probe hits.

Downstream (consumes this area):

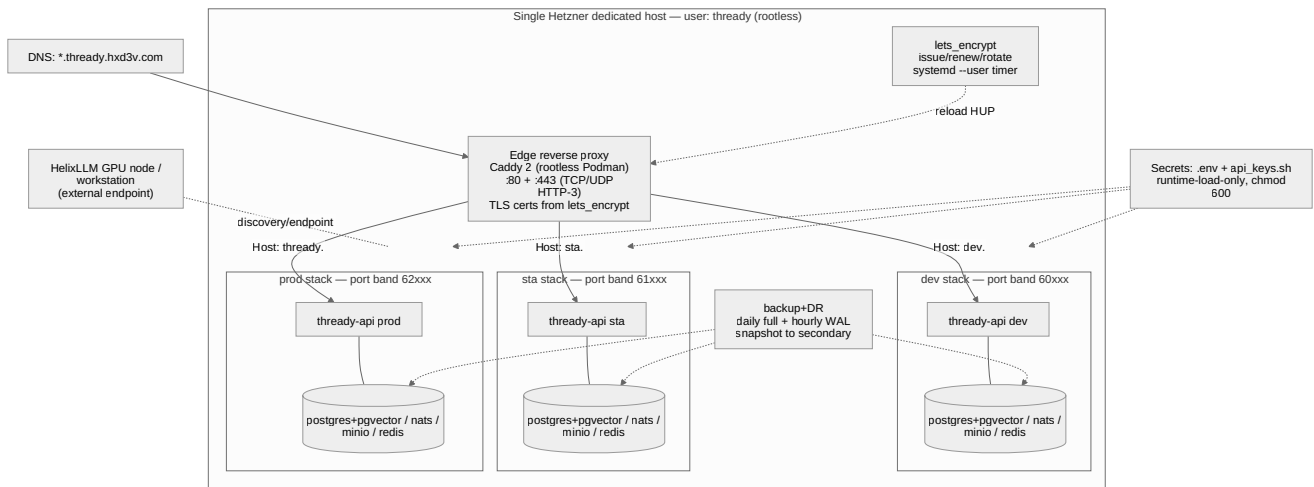
- **development** — the ATM-NNN workable items that build the new submodules (Download Manager, User Service, Max adapter, etc.) whose containers this topology reserves slots for; the local git-hook enforcement defined in `secrets-and-config.md`.
- **testing** — the chaos / DR / scaling tests (`§11.4.27`) that validate the `backup-dr.md` RPO/RTO targets and the `deploy-and-rollback.md` rollback gate.

3. In-house modules this area binds to

Every module below is named in the decision matrix (`§0.2`). Interfaces were read at source (`gh repo view` / shallow clone) during authoring — **do not invent APIs** `[CONVENTIONS §1]`.

Module	Repo	Role in deployment	Verified surface used
Containers	<code>vasic-digital/containers</code>	Rootless compose orchestration, boot, health, discovery, remote deploy	<code>pkg/compose.ComposeOrchestrator</code> , <code>pkg/boot.BootManager</code> , <code>pkg/health.HealthChecker</code> , <code>pkg/endpoint.ServiceEndpoint</code> , <code>pkg/serviceregistry.ServiceRegistry</code> , <code>cmd/deploy-stack</code>
Let's Encrypt	<code>vasic-digital/lets_encrypt</code>	ACME issuance/renewal/rotation; atomic deploy-hook + rollback	<code>scripts/{issue,renew,rotate,revoke,deploy-hook}.sh</code> , <code>config/lets_encrypt.conf</code> , <code>systemd/*</code> , <code>api/le-api</code>
Port prefix	<code>vasic-digital/port_prefix</code>	Deterministic per-env host-port bands	<code>portprefix.Exposed(prefix, internalPort, taken) (int, error)</code>
mDNS	<code>vasic-digital/mdns</code>	LAN service announcement/discovery	<code>Announce</code> / <code>Browse</code>
Discovery	<code>vasic-digital/discovery</code>	Network/service scanning	<code>pkg/scanner.Scanner</code>
Observability	<code>vasic-digital/observability</code>	Health aggregation, metrics, tracing	<code>pkg/health.{Checker,Aggregator,Report}</code> , <code>pkg/metrics</code>
HTTP/3	<code>vasic-digital/http3</code>	QUIC transport for API + edge	<code>http3.Config</code> / <code>http3.Server</code>
Security	<code>digital.vasic.security</code>	AES-256-GCM sealed key store for secrets at rest	<code>pkg/securestorage</code>

4. Deployment overview diagram



Explanation (for readers/models that cannot see the diagram). The entire system lives on a single Hetzner dedicated host and runs entirely as the unprivileged `thready` user — there is no Docker daemon and no `sudo` in the runtime path [CONSTITUTION §11.4.76/161]. Public DNS points the wildcard `*.thready.hxd3v.com` (and the apex `thready.hxd3v.com`) at the host's single public IP.

A single **rootless edge reverse proxy** (Caddy 2, itself a rootless Podman container) is the only process bound to the public ports 80 and 443 (443 on both TCP and UDP so HTTP/3 works). It terminates TLS using certificates that the `lets_encrypt` module issues and installs, then routes by HTTP `Host` header to the correct environment: `dev.` → the dev stack, `sta.` → the staging stack, `thready.` (apex) → the production stack.

Each of the three environments is an **independent Podman Compose project** with its own Postgres+pgvector, NATS JetStream, MinIO object store and Redis cache — nothing is shared across environments, which is what “fully separated” means (§8.2). The three stacks never collide on the host because each is assigned a disjoint **host-port band** via the `port_prefix` module (dev = `60xxx`, sta = `61xxx`, prod = `62xxx`); the details are in `service-discovery-ports.md`. The `lets_encrypt` renewal runs as a `systemd --user` timer and, on a successful renew, atomically installs the new cert and sends the edge proxy a `HUP` reload with zero dropped connections. The **backup/DR** subsystem takes a daily full backup plus hourly PostgreSQL WAL increments of all three stacks and ships snapshots to a secondary store, meeting the RPO ≈ 1 h / RTO ≈ 4 h targets. Because the operator workstation holds the 32 GB GPU, **HelixLLM** is reached as an *external endpoint* (via the containers `discovery` / `endpoint` layer) rather than being packaged inside the per-environment compose files. Finally, every stack loads its secrets at runtime only, from gitignored `.env` and `api_keys.sh` files with `chmod 600`, never from a tracked file and never into logs [CONSTITUTION §11.4.10].

5. Document map

Document	What it specifies
container-topology.md	The full per-environment service inventory, container images, networks, volumes, resource limits, and the BUILD-NEW placeholders
podman-compose.md	Rootless Podman + <code>podman-compose</code> runtime, the <code>containers</code> boot/orchestrator API, compose file layout, health gating
compose-files.md	The complete, copy-paste-ready rootless Podman Compose YAML for the edge + dev/sta/prod, the <code>buildnew</code> profile, healthchecks, loopback bindings, and the per-env render step
environments.md	The three-environment separation model, subdomain routing, per-env config, promotion flow
tls-lets-encrypt.md	ACME issuance (HTTP-01/DNS-01), renewal timer, atomic deploy-hook + risk-free rollback, per-subdomain certs
deploy-and-rollback.md	The deploy pipeline (bash + Go health gate), image-tag pinning, expand-contract migrations, rollback
backup-dr.md	Daily full + hourly WAL backups, asset snapshots, RPO/RTO, the restore runbook, chaos validation
service-discovery-ports.md	<code>port_prefix</code> bands, <code>discovery / mdns</code> , <code>serviceregistry</code> , the deterministic port plan
hetzner-provisioning.md	Root bootstrap → <code>thready</code> user, rootless Podman setup, firewall, linger, systemctl, first deploy
secrets-and-config.md	Secret sources, load precedence, <code>chmod</code> , leak-audit, local git-hook enforcement (no server CI)
operations-runbook.md	Day-2 operations: on-call quick card, routine ops, incident-triage, per-failure playbooks (service down, cert, WAL, disk, reboot, stuck deploy, leak), maintenance cadence
monitoring-observability.md	The Q42/Q43 observability deployment: Prometheus/Grafana/OTel-Jaeger/ClickHouse per env, SLOs, the complete alert-rule set (service-down, edge, cert-expiry, latency, resource), alert routing, dashboards, reproduce-first alert test
cost-and-capacity.md	Monthly run-cost quantification (<code>\$16.3</code>): Hetzner host + object tier + backup secondary + GPU (external), the capacity envelope, three cost scenarios, and the subscription+metered billing-offset model

6. Gap-register items addressed by this area

From `helix_thready_subsystem_gaps_and_improvements.md`. Each is tagged `[GAP: ...]` at the point of treatment in the relevant document.

Gap	Where addressed	Treatment
#1 HelixLLM HashEmbedder trap (P0)	secrets-and-config.md, environments.md	Mandatory <code>HELIX_EMBEDDING_PROVIDER=llama</code> env var enforced per environment; deploy gate fails if unset for a semantic workload
#3.2 database/ storage — no partitioning; MinIO signed-URL parity (P1)	backup-dr.md, container-topology.md	Self-hosted MinIO on Hetzner validated for signed URLs; time-partition-aware backup plan; tracked item
#19 docs_chain SKIPs without pandoc/ weasyprint (P2)	hetzner-provisioning.md	Provision <code>pandoc</code> + <code>weasyprint</code> in the host toolchain so md→HTML/PDF siblings generate
#12 CI-equivalent gating (P1, cross-cutting)	secrets-and-config.md, deploy-and-rollback.md	Local git-hooks (secret-scan pre-commit, full-suite pre-tag retest) replace forbidden server CI
#12 Decoupling / anti-bluff sweep (P1)	deploy-and-rollback.md	Deploy verification asserts real health (not stub) before promotion; DEV-only default creds in the <code>containers</code> compose are overridden
#20 BUILD-NEW subsystems	container-topology.md	Their containers are declared as <i>placeholders</i> and clearly marked not-yet-working until built
#17 LLMsVerifier port discrepancy (:7061 vs :8080) (P2)	service-discovery-ports.md	<code>HELIX_LLM_VERIFIER_URL</code> pinned explicitly per env to the port LLMsVerifier actually serves; endpoint marked <code>Required</code> only if in the fallback chain, else skipped

7. Open items tracked by this area

ID	Item	Status	Where
[OPEN: dns-provider]	acme.sh DNS-01 hook for <code>hxd3v.com</code>	Narrowed (Rev 3) — concrete <code>dns_loopia</code> path documented (verified config-example provider + reserved <code>LOOPIA_*</code> keys); HTTP-01 is the working default; closes on operator registrar confirmation	tls-lets-encrypt.md §4.1
[OPEN: host-sizing]	Exact Hetzner SKU + object-tier backing	Narrowed (Rev 3) — candidate AX/SX SKU families + object-tier split table added; specific model confirmed at purchase after load tests	hetzner-provisioning.md §1.1
[OPEN: secondary-store]	Backup secondary target (Storage Box vs 2nd MinIO)	Proposed default (Storage Box); both meet physical-separation	backup-dr.md
[OPEN: buildnew-images]	Images for BUILD-NEW services do not exist yet	Reserved via the <code>buildnew</code> compose profile so a default <code>up</code> excludes them	container-topology.md, compose-files.md §6
[OPEN: pgvector-image]	<code>pgvector/pgvector:pg17</code> vs <code>postgres:17-alpine</code> + extension	Both satisfy the topology; owned by the database area	compose-files.md §10
[OPEN: alerting-route]	Where alerts page (email/Telegram/PagerDuty)	The complete rule set (<code>what</code> fires, with <code>severity</code> / <code>page</code> labels) is now defined; <code>who</code> is paged is an observability-wiring choice	monitoring-observability.md §6, operations-runbook.md §13
[OPEN: pricing]	Subscription tier prices + metered rates	This pack quantifies COGS (<code>\$16.3</code>); the price card is a product/billing decision owned by the User/Billing service	cost-and-capacity.md §6

8. Verified vs assumed — reading guide

- **VERIFIED** facts are drawn from module source read during authoring (`containers` , `lets_encrypt` , `port_prefix` , `mdns` , `discovery` , `observability`) or from the authoritative research documents. They are stated plainly and cite the module/section.
- **ASSUMPTIONS / DEFAULTS** are tagged `[DEFAULT — adjustable]` — enterprise defaults the operator may override (port-band prefixes, backup retention, resource limits, Caddy-vs-nginx edge).

- Where a module is a **scaffold/stub** per the gap register, this pack **never claims it works**: its container is a declared placeholder with a `[GAP: ...]` tag and a plan to close it.
-

Made with love ♥ by Helix Development.